ONEDRAFT

CAD-AI — Aria Powered

Runtime Orchestrator

Version 0.1

Status: Implementation Specification
Runtime Layer: Orchestrated Services / Workers
API Boundary: `/api/v1`
Primary Execution Model: Event-Driven / Job-Oriented
State Authority: PostgreSQL Project Model
AI Layer: Aria Powered
Execution Principle: Deterministic orchestration of versioned OneDraft operations

---

1. RUNTIME ORCHESTRATION PRINCIPLE

The Runtime Orchestrator is the execution layer of OneDraft.

It does not replace the Project Model.

It does not replace the Geometry Engine.

It does not replace the Compliance Engine.

It does not replace the Validation Engine.

It does not replace the Revision System.

It does not replace the Documentation and Drawing Generation Engine.

Instead, the Runtime Orchestrator coordinates these established subsystems into executable workflows.

The architecture therefore becomes:

```
CLIENT
  ↓
API
  ↓
RUNTIME ORCHESTRATOR
  ↓
DOMAIN SERVICES
  ├── PROJECT MODEL
  ├── ARIA
  ├── GEOMETRY
  ├── COMPLIANCE
  ├── VALIDATION
```

```
  ├── REVISION
  └── DOCUMENTATION
  ↓
PERSISTENCE / OBJECT STORAGE
```

The Runtime Orchestrator is responsible for deciding **what executes, when it executes, under which version, with which dependencies, and what happens when execution succeeds or fails.**

---

2. RUNTIME SYSTEM OF RECORD

The Runtime Orchestrator shall not become an independent source of project truth.

The authoritative hierarchy remains:

```
PROJECT MODEL
  ↓
MODEL VERSION
  ↓
REVISION
  ↓
CODE MANIFEST
  ↓
VALIDATION
  ↓
DOCUMENTATION
  ↓
ACCEPTANCE
```

Runtime state records execution against these authoritative objects.

The orchestrator may create:

```
jobs
tasks
workflow executions
events
locks
dependencies
execution results
failure records
retry records
```

but these records describe execution rather than replacing domain state.

---

3. RUNTIME RESPONSIBILITIES

The Runtime Orchestrator shall provide:

workflow execution

job scheduling

task dispatch

dependency resolution

worker coordination

execution state management

retry handling

failure handling

cancellation

timeout enforcement

version verification

idempotency enforcement

event publication

progress reporting

resource coordination

workflow recovery

execution observability

---

4. RUNTIME NON-RESPONSIBILITIES

The Runtime Orchestrator shall not independently determine:

```
architectural geometry
building-code requirements
design intent
model topology
drawing standards
customer acceptance
professional approval
```

Those decisions remain within their respective domain services.

The orchestrator executes authorized operations against those services.

---

5. EXECUTION MODEL

Every substantial OneDraft operation shall be represented as a runtime job.

Conceptually:

```
REQUEST
 ↓
JOB
 ↓
WORKFLOW
 ↓
TASKS
 ↓
DOMAIN SERVICES
 ↓
RESULTS
 ↓
VALIDATION
 ↓
FINALIZATION
```

A job may contain one or many tasks.

A task represents one executable unit.

A workflow represents the dependency graph connecting tasks.

---

6. JOB IDENTIFIER

Every runtime job receives:

```
job_id UUID
```

The identifier remains stable throughout the execution lifecycle.

External clients use the `job_id` to retrieve execution state.

---

7. JOB TABLE

The initial persistence layer shall introduce:

```
system.jobs
```

Fields:

```
id UUID PRIMARY KEY
project_id UUID
organization_id UUID
job_type VARCHAR(64) NOT NULL
workflow_type VARCHAR(64)
requested_by UUID
source_revision_id UUID
source_model_version_id UUID
source_code_manifest_id UUID
```

```
status VARCHAR(32) NOT NULL
priority INTEGER DEFAULT 0
progress NUMERIC
request_payload JSONB
result_payload JSONB
error_payload JSONB
idempotency_key VARCHAR(255)
created_at TIMESTAMPTZ NOT NULL
started_at TIMESTAMPTZ
completed_at TIMESTAMPTZ
updated_at TIMESTAMPTZ NOT NULL
```

---

8. JOB STATUS

Initial statuses:

```
QUEUED
RUNNING
WAITING
VALIDATING
COMPLETED
FAILED
CANCELLED
EXPIRED
```

The state transition must be controlled by the runtime state machine.

---

9. JOB STATE MACHINE

Valid transitions include:

```
QUEUED
  ↓
RUNNING
  ↓
COMPLETED
```

or:

```
QUEUED
  ↓
RUNNING
  ↓
FAILED
```

or:

```
QUEUED
  ↓
CANCELLED
```

or:

```
RUNNING
  ↓
WAITING
  ↓
RUNNING
```

The system shall reject invalid state transitions.

---

10. TASK MODEL

A job is decomposed into executable tasks.

Example:

```
DOCUMENT_GENERATION
          │
          ├── LOAD_MODEL
          ├── RESOLVE_VIEWS
          ├── GENERATE_GEOMETRY
          ├── RUN_COORDINATION
          ├── GENERATE_DRAWINGS
          ├── GENERATE_SCHEDULES
          ├── VALIDATE_DOCUMENTATION
          ├── PACKAGE_DOCUMENTS
          └── HASH_PACKAGE
```

Each task has its own execution state.

---

11. TASK TABLE

`system.job_tasks`

Fields:

```
id UUID PRIMARY KEY
job_id UUID NOT NULL
task_type VARCHAR(64) NOT NULL
sequence INTEGER
status VARCHAR(32) NOT NULL
attempt_count INTEGER DEFAULT 0
max_attempts INTEGER DEFAULT 3
priority INTEGER DEFAULT 0
worker_type VARCHAR(64)
input_payload JSONB
result_payload JSONB
error_payload JSONB
started_at TIMESTAMPTZ
completed_at TIMESTAMPTZ
created_at TIMESTAMPTZ NOT NULL
```

```
updated_at TIMESTAMPTZ NOT NULL
```

---

12. TASK DEPENDENCIES

```
system.job_dependencies
```

Fields:

```
id UUID PRIMARY KEY
job_id UUID NOT NULL
task_id UUID NOT NULL
depends_on_task_id UUID NOT NULL
dependency_type VARCHAR(32) NOT NULL
created_at TIMESTAMPTZ NOT NULL
```

Primary dependency types:

```
REQUIRED
OPTIONAL
CONDITIONAL
```

---

13. DIRECTED EXECUTION GRAPH

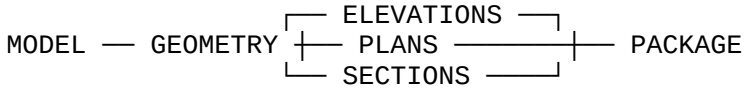
Runtime workflows shall be represented as directed acyclic graphs where possible.

Example:

```
LOAD MODEL
    ↓
RESOLVE CONSTRAINTS
    ↓
GEOMETRY
    ↓
VALIDATION
    ↓
DRAWINGS
    ↓
DOCUMENT PACKAGE
```

Independent operations may execute concurrently.

For example:



---

14. WORKFLOW DEFINITIONS

Workflow definitions shall be versioned.

```
system.workflow_definitions
```

Fields:

```
id UUID PRIMARY KEY
name VARCHAR(128) NOT NULL
version INTEGER NOT NULL
definition JSONB NOT NULL
status VARCHAR(32) NOT NULL
created_at TIMESTAMPTZ NOT NULL
updated_at TIMESTAMPTZ NOT NULL
```

A workflow definition is immutable after activation.

Changes create a new version.

---

15. WORKFLOW EXECUTIONS

```
system.workflow_executions
```

Fields:

```
id UUID PRIMARY KEY
workflow_definition_id UUID NOT NULL
job_id UUID NOT NULL
status VARCHAR(32) NOT NULL
current_step VARCHAR(128)
progress NUMERIC
context JSONB
started_at TIMESTAMPTZ
completed_at TIMESTAMPTZ
created_at TIMESTAMPTZ NOT NULL
updated_at TIMESTAMPTZ NOT NULL
```

---

16. RUNTIME CONTEXT

Every workflow receives an execution context containing:

```
organization_id
project_id
revision_id
model_version_id
code_manifest_id
job_id
workflow_id
```

```
request_id
actor_id
```

Additional subsystem-specific context may be added.

The context shall never silently change the source model version.

---

17. VERSION PINNING

A runtime operation must pin its source state.

For model-dependent operations:

```
project_id
revision_id
model_version_id
```

For compliance-dependent operations:

```
code_manifest_id
```

For documentation operations:

```
revision_id
model_version_id
```

This ensures reproducibility.

---

18. STALE EXECUTION PROTECTION

Before committing a result, the orchestrator verifies that the source state remains valid.

Example:

```
JOB CREATED AGAINST REVISION 17
        ↓
REVISION 18 CREATED
        ↓
JOB COMPLETES
        ↓
VERSION CHECK
        ↓
RESULT CANNOT AUTOMATICALLY BECOME CURRENT
```

The result may be retained as historical execution output but must not silently overwrite newer state.

---

19. COMMAND EXECUTION PIPELINE

An Aria command shall execute through:

```
ARIA REQUEST
 ↓
AUTHORIZATION
 ↓
COMMAND VALIDATION
 ↓
CURRENT REVISION CHECK
 ↓
CONSTRAINT CHECK
 ↓
JOB CREATION
 ↓
TASK DISPATCH
 ↓
DOMAIN OPERATION
 ↓
GEOMETRY
 ↓
VALIDATION
 ↓
REVISION FINALIZATION
 ↓
EVENT
```

---

20. ARIA EXECUTION BOUNDARY

Aria does not directly manipulate infrastructure.

Aria produces an executable intent.

Example:

```
USER INTENT
 ↓
ARIA
 ↓
STRUCTURED COMMAND
 ↓
COMMAND VALIDATOR
 ↓
RUNTIME ORCHESTRATOR
 ↓
PROJECT MODEL SERVICE
```

The structured command becomes the controlled interface between intelligence and execution.

---

21. COMMAND VALIDATION

Before execution, the runtime layer verifies:

```
command schema
target objects
project membership
authorization
expected revision
required constraints
operation availability
resource availability
```

Invalid commands do not enter execution.

---

22. COMMAND IDEMPOTENCY

Every externally submitted command shall support:

```
Idempotency-Key
```

The runtime layer determines whether an equivalent command has already been accepted.

A retry must not create duplicate model mutations.

---

23. TASK IDEMPOTENCY

Tasks shall additionally contain an execution identity.

Conceptually:

```
job_id
task_id
attempt_number
```

A worker must be able to safely determine whether a task has already completed before applying an external side effect.

---

24. WORKER ARCHITECTURE

Runtime execution shall use specialized workers.

Initial worker classes:

```
API WORKER
```

```
ORCHESTRATION WORKER
MODEL WORKER
GEOMETRY WORKER
COMPLIANCE WORKER
VALIDATION WORKER
DRAWING WORKER
DOCUMENT WORKER
EXPORT WORKER
```

Workers communicate through controlled service interfaces.

---

25. WORKER ISOLATION

Workers shall not directly modify unrelated subsystem state.

For example:

```
GEOMETRY WORKER
```

may calculate geometry but shall not directly modify:

```
billing
authorization
customer acceptance
```

Domain ownership remains explicit.

---

26. QUEUE MODEL

The runtime layer shall use durable queues.

Conceptually:

```
REQUEST QUEUE
  ↓
JOB QUEUE
  ↓
TASK QUEUE
  ↓
WORKER
```

Queues shall preserve tasks across temporary worker failures.

---

27. QUEUE PRIORITIES

Initial priorities:

```
CRITICAL
HIGH
NORMAL
LOW
BACKGROUND
```

Customer-facing document generation may receive higher priority than background analytics.

---

28. RESOURCE CLASSES

Jobs shall declare resource requirements where appropriate.

Example:

```
CPU
MEMORY
GPU
GEOMETRY_ENGINE
DOCUMENT_ENGINE
COMPLIANCE_ENGINE
```

The scheduler uses these requirements to determine worker placement.

---

29. CONCURRENCY

Independent tasks may execute concurrently.

Dependent tasks shall wait for prerequisite completion.

Example:

```
PLAN
ELEVATION
SECTION
```

may generate concurrently once the required model state is available.

However:

```
MODEL MUTATION
```

must complete before dependent drawing generation begins.

---

30. PROJECT MUTATION LOCK

The runtime system shall prevent conflicting model mutations.

Conceptually:

```
PROJECT
  ↓
MUTATION LOCK
  ↓
COMMAND
  ↓
REVISION
  ↓
LOCK RELEASE
```

This does not prohibit concurrent read operations.

---

31. REVISION OPTIMISTIC CONCURRENCY

The preferred model mutation mechanism is optimistic concurrency.

A command specifies:

```
expected_revision = 17
```

The server verifies:

```
current_revision == 17
```

If the project is already at Revision 18:

```
REVISION_CONFLICT
```

is returned.

---

32. LONG-RUNNING WORKFLOW LOCKS

Long-running workflows shall not hold database transactions open.

Instead:

```
DATABASE TRANSACTION
  ↓
PERSIST JOB
  ↓
COMMIT
  ↓
WORKER EXECUTION
  ↓
FINALIZATION TRANSACTION
```

This prevents long-lived database locks.

---

33. RETRY POLICY

Transient failures may be retried.

Initial policy:

```
attempt 1
 ↓
short delay
 ↓
attempt 2
 ↓
exponential backoff
 ↓
attempt 3
```

Permanent domain failures shall not be blindly retried.

---

34. RETRY CLASSIFICATION

Errors shall be classified as:

```
TRANSIENT
PERMANENT
CONFLICT
AUTHORIZATION
VALIDATION
RESOURCE
TIMEOUT
DEPENDENCY
VERSION
SYSTEM
```

Only retryable classes enter automatic retry.

---

35. EXPONENTIAL BACKOFF

Retry delays shall use controlled exponential backoff.

Conceptually:

```
delay = base × 2^attempt
```

with a maximum retry interval.

Randomized jitter should be applied to prevent synchronized retry storms.

---

36. DEAD-LETTER QUEUE

Tasks exceeding their retry policy shall enter:

```
DEAD_LETTER
```

The original execution context and failure history shall remain available.

Dead-letter tasks must not disappear from the system.

---

37. FAILURE RECORD

```
system.job_failures
```

Fields:

```
id UUID PRIMARY KEY
job_id UUID NOT NULL
task_id UUID
failure_type VARCHAR(64) NOT NULL
error_code VARCHAR(128)
message TEXT
details JSONB
retryable BOOLEAN NOT NULL
attempt INTEGER
occurred_at TIMESTAMPTZ NOT NULL
```

---

38. TIMEOUT MANAGEMENT

Every task class shall have a defined timeout.

Example:

```
MODEL_OPERATION
GEOMETRY_OPERATION
VALIDATION
DRAWING_GENERATION
DOCUMENT_GENERATION
```

The timeout is enforced by the runtime layer rather than relying solely on the worker.

---

39. CANCELLATION

Jobs may be cancelled while they are:

```
QUEUED
RUNNING
WAITING
```

Cancellation must be cooperative where execution has already started.

A completed mutation cannot be retroactively cancelled.

A compensating operation or new revision is required when reversal is appropriate.

---

40. JOB PROGRESS

The runtime layer shall expose normalized progress.

Example:

```
0
25
50
75
100
```

Progress may be calculated from weighted tasks.

Example:

```
MODEL LOAD       10%
GEOMETRY         25%
VALIDATION       20%
DRAWINGS         25%
PACKAGING        20%
```

---

41. PROGRESS EVENTS

Workers shall publish execution events such as:

```
JOB_STARTED
TASK_STARTED
TASK_PROGRESS
TASK_COMPLETED
TASK_FAILED
WORKFLOW_WAITING
VALIDATION_STARTED
VALIDATION_COMPLETED
DOCUMENT_GENERATION_STARTED
DOCUMENT_GENERATION_COMPLETED
```

```
JOB_COMPLETED
JOB_FAILED
```

---

42. RUNTIME EVENTS

`system.runtime_events`

Fields:

```
id UUID PRIMARY KEY
job_id UUID
task_id UUID
project_id UUID
event_type VARCHAR(128) NOT NULL
payload JSONB NOT NULL
timestamp TIMESTAMPTZ NOT NULL
```

Runtime events are append-only.

---

43. EVENT CORRELATION

Every request shall receive:

`request_id`

Every job shall receive:

`job_id`

Every workflow shall receive:

`workflow_execution_id`

Every task shall receive:

`task_id`

These identifiers allow complete execution tracing.

---

44. DISTRIBUTED CORRELATION

A request may therefore be traced:

```
REQUEST_ID
  ↓
JOB_ID
```

```
 ↓
WORKFLOW_ID
 ↓
TASK_ID
 ↓
WORKER
 ↓
DOMAIN SERVICE
 ↓
RESULT
```

This becomes the runtime observability chain.

---

45. SERVICE CONTRACT

Domain services shall expose controlled operations.

Conceptually:

```
ProjectModelService
GeometryService
ComplianceService
ValidationService
DocumentationService
RevisionService
```

The orchestrator invokes these interfaces rather than manipulating their databases directly.

---

46. SERVICE RESULT MODEL

Domain service results shall return structured responses.

Example:

```
{
  "status": "SUCCESS",
  "project_id": "uuid",
  "revision_id": "uuid",
  "model_version_id": "uuid",
  "affected_objects": [],
  "result": {}
}
```

Failures use structured error codes.

---

47. DOMAIN TRANSACTION BOUNDARY

Each domain service remains responsible for its own transactional integrity.

The Runtime Orchestrator coordinates transactions across services through workflow state rather than attempting one global database transaction.

---

48. SAGA-STYLE EXECUTION

Long-running cross-service workflows shall use compensating actions where necessary.

Conceptually:

```
TASK A
 ↓
TASK B
 ↓
TASK C
 ↓
FAILURE
 ↓
COMPENSATING ACTIONS
```

The objective is controlled recovery rather than distributed two-phase database transactions.

---

49. MODEL MUTATION FINALIZATION

A model mutation workflow shall follow:

```
COMMAND
 ↓
PROPOSAL
 ↓
VALIDATION
 ↓
MODEL MUTATION
 ↓
GEOMETRY
 ↓
VALIDATION
 ↓
REVISION
 ↓
DRAWING IMPACT
```

Only after successful finalization does the new revision become current.

---

50. DRAWING REGENERATION ORCHESTRATION

When a model change affects drawings:

```
MODEL CHANGE
 ↓
IMPACT ANALYSIS
 ↓
AFFECTED VIEWS
 ↓
AFFECTED DRAWINGS
 ↓
DRAWING GENERATION
 ↓
DOCUMENT VALIDATION
```

Unaffected drawings may remain associated with their existing compatible model state where the documentation engine determines that they remain valid.

---

51. REDLINE ORCHESTRATION

A redline workflow becomes:

```
REDLINE CREATED
 ↓
ARIA INTERPRETATION
 ↓
COMMAND PROPOSAL
 ↓
COMMAND VALIDATION
 ↓
MODEL MUTATION
 ↓
GEOMETRY
 ↓
DRAWING UPDATE
 ↓
VALIDATION
 ↓
REDLINE RESOLUTION
```

A redline shall not be marked resolved merely because a command was generated.

Resolution requires successful implementation.

---

52. COMPLIANCE WORKFLOW

The compliance execution path shall be:

```
PROJECT
 ↓
JURISDICTION
 ↓
CODE MANIFEST
 ↓
APPLICABLE RULES
 ↓
MODEL ANALYSIS
 ↓
VALIDATION
 ↓
RESULTS
```

The runtime layer coordinates this process.

The Compliance Engine remains responsible for regulatory interpretation and rule evaluation.

---

53. VALIDATION WORKFLOW

Validation shall execute as:

```
REVISION
 ↓
CODE MANIFEST
 ↓
VALIDATION RUN
 ↓
RULE EXECUTION
 ↓
RESULTS
 ↓
SUMMARY
```

Validation may execute multiple categories concurrently.

---

54. DOCUMENT GENERATION WORKFLOW

The complete document workflow becomes:

```
PROJECT
 ↓
REVISION
 ↓
```

```
MODEL VERSION
 ↓
VIEWS
 ↓
DRAWINGS
 ↓
SCHEDULES
 ↓
ANNOTATIONS
 ↓
DOCUMENT VALIDATION
 ↓
PACKAGE
 ↓
HASH
 ↓
ACCEPTANCE
```

The Runtime Orchestrator coordinates the sequence.

---

55. FULL PROJECT EXECUTION

The principal OneDraft runtime workflow shall be:

```
CREATE PROJECT
 ↓
CAPTURE REQUIREMENTS
 ↓
BUILD PROJECT MODEL
 ↓
ESTABLISH JURISDICTION
 ↓
CREATE CODE MANIFEST
 ↓
GENERATE MODEL
 ↓
GENERATE GEOMETRY
 ↓
RUN COMPLIANCE
 ↓
RUN VALIDATION
 ↓
GENERATE DOCUMENTATION
 ↓
CUSTOMER REDLINES
 ↓
ARIA COMMANDS
 ↓
MODEL REVISION
 ↓
REGENERATE AFFECTED DOCUMENTATION
 ↓
REVALIDATE
 ↓
FINAL PACKAGE
```

```
 ↓
ACCEPTANCE
```

---

56. EVENT-DRIVEN EXECUTION

The runtime system shall support events as workflow triggers.

Examples:

```
PROJECT_CREATED
REVISION_CREATED
REDLINE_CREATED
COMMAND_APPROVED
GEOMETRY_COMPLETED
VALIDATION_FAILED
VALIDATION_COMPLETED
DRAWING_GENERATED
DOCUMENT_PACKAGE_GENERATED
CUSTOMER_ACCEPTED
```

Events may initiate downstream jobs.

---

57. EVENT OUTBOX

Domain services shall use an outbox pattern where reliable event publication is required.

Conceptually:

```
DOMAIN TRANSACTION
 ↓
STATE CHANGE
 +
OUTBOX EVENT
 ↓
COMMIT
 ↓
EVENT PUBLISHER
 ↓
RUNTIME
```

This prevents state changes from being committed without their associated events being published.

---

58. OUTBOX TABLE

```
system.outbox_events
```

Fields:

```
id UUID PRIMARY KEY
aggregate_type VARCHAR(64) NOT NULL
aggregate_id UUID NOT NULL
event_type VARCHAR(128) NOT NULL
payload JSONB NOT NULL
status VARCHAR(32) NOT NULL
attempt_count INTEGER DEFAULT 0
created_at TIMESTAMPTZ NOT NULL
published_at TIMESTAMPTZ
```

---

59. EVENT DELIVERY

Initial delivery states:

```
PENDING
PUBLISHED
FAILED
```

Failed events remain persisted until successfully processed or explicitly quarantined.

---

60. WORKFLOW TRIGGERS

A workflow trigger contains:

```
event_type
condition
workflow_type
workflow_version
priority
```

Example:

```
EVENT:
REVISION_CREATED

CONDITION:
revision.status = FINALIZED

WORKFLOW:
DRAWING_REGENERATION
```

---

61. CONDITIONAL EXECUTION

The orchestrator shall support conditions.

Example:

```
IF geometry_changed
    RUN geometry
ELSE
    SKIP geometry
```

Another example:

```
IF validation.errors > 0
    STOP FINALIZATION
ELSE
    CONTINUE
```

---

62. HUMAN-IN-THE-LOOP BOUNDARY

Certain operations may require explicit human approval.

The workflow may therefore enter:

```
WAITING_FOR_APPROVAL
```

Examples include:

```
customer acceptance
professional review
high-impact design change
final document approval
```

The orchestrator resumes after the required approval event is received.

---

63. APPROVAL DOES NOT MUTATE HISTORY

Approval records are appended to the existing revision workflow.

Approval does not overwrite prior state.

The sequence remains:

```
REVISION
  ↓
REVIEW
  ↓
APPROVAL
  ↓
ACCEPTANCE
```

---

64. EXECUTION SECURITY

The runtime layer shall enforce:

```
authentication
authorization
organization isolation
project isolation
scope verification
command validation
worker identity
service identity
```

Workers shall receive only the credentials required for their assigned operation.

---

65. TENANCY ISOLATION

Every project-scoped job shall carry:

```
organization_id
project_id
```

Workers must verify tenancy before executing project data operations.

Cross-organization access is prohibited unless explicitly authorized through a controlled service boundary.

---

66. SECRET MANAGEMENT

Runtime jobs shall never persist raw credentials.

Secrets belong in the deployment secret-management layer.

Job payloads may contain:

```
provider_reference
credential_reference
```

but not plaintext secrets.

---

67. FILE AND OBJECT STORAGE

Large artifacts shall not be stored directly in job payloads.

Examples:

```
geometry files
PDF packages
rendered drawings
source documents
code artifacts
large model exports
```

shall use object storage.

The runtime record stores references.

---

68. ARTIFACT REGISTRY

Generated artifacts shall have stable identifiers.

Conceptually:

```
artifact_id
project_id
revision_id
model_version_id
job_id
storage_reference
content_hash
mime_type
size
created_at
```

This creates a direct relationship between runtime execution and generated files.

---

69. ARTIFACT IMMUTABILITY

Finalized artifacts shall be immutable.

A changed artifact receives a new version.

The system must never silently replace a finalized document package.

---

70. HASH VERIFICATION

Generated artifacts shall be hashed.

Initial algorithm:

```
SHA-256
```

The hash shall be stored with the artifact metadata.

This supports integrity verification and reproducibility.

---

71. EXECUTION REPRODUCIBILITY

A completed workflow shall retain enough information to identify:

```
input revision
model version
code manifest
workflow version
task versions
generated artifacts
validation results
execution events
```

The objective is reproducible project execution.

---

72. RUNTIME CONFIGURATION

Runtime configuration shall be externalized.

Configuration includes:

```
worker concurrency
queue names
timeouts
retry limits
resource limits
feature flags
storage locations
service endpoints
```

Configuration changes shall not require rewriting domain code.

---

73. FEATURE FLAGS

Feature flags may control:

```
experimental geometry workers
new validation rules
new drawing generators
new workflow versions
AI capabilities
export formats
```

Production workflows must record the relevant execution configuration when reproducibility requires it.

---

74. OBSERVABILITY

The Runtime Orchestrator shall expose:

```
logs
metrics
traces
job state
task state
worker state
queue state
failure state
```

Observability data shall be correlated through the runtime identifiers.

---

75. METRICS

Initial metrics include:

```
jobs_created_total
jobs_completed_total
jobs_failed_total
jobs_cancelled_total
task_execution_duration
workflow_duration
queue_depth
worker_utilization
retry_count
validation_duration
geometry_duration
drawing_generation_duration
document_generation_duration
```

---

76. HEALTH CHECKS

Each runtime service shall provide:

```
health
readiness
liveness
```

Health checks shall distinguish between:

```
process available
service operational
dependency available
```

---

77. WORKER HEARTBEATS

Long-running workers shall periodically publish heartbeats.

A heartbeat contains:

```
worker_id
job_id
task_id
timestamp
progress
```

The orchestrator uses heartbeats to detect stalled workers.

---

78. STALLED TASK RECOVERY

If a worker stops responding:

```
HEARTBEAT TIMEOUT
 ↓
TASK MARKED STALLED
 ↓
WORKER INSTANCE INVALIDATED
 ↓
TASK RECOVERY
 ↓
RETRY
```

Recovery must respect idempotency.

---

79. WORKER REGISTRY

```
system.workers
```

Fields:

```
id UUID PRIMARY KEY
worker_type VARCHAR(64) NOT NULL
hostname VARCHAR(255)
status VARCHAR(32) NOT NULL
capacity JSONB
last_heartbeat TIMESTAMPTZ
registered_at TIMESTAMPTZ NOT NULL
```

Workers may be ephemeral.

---

80. EXECUTION CAPABILITIES

Workers shall advertise capabilities.

Example:

```
GEOMETRY_3D
BIM_EXPORT
PDF_GENERATION
SVG_GENERATION
CODE_VALIDATION
IMAGE_RENDERING
```

The scheduler matches task requirements to worker capabilities.

---

81. RUNTIME API

The API shall expose:

```
POST /api/v1/jobs
GET  /api/v1/jobs/{job_id}
POST /api/v1/jobs/{job_id}/cancel
GET  /api/v1/jobs/{job_id}/events
```

Project workflows may expose higher-level endpoints while internally creating runtime jobs.

---

82. JOB CREATION REQUEST

Example:

```
POST /api/v1/jobs
```

Request:

```
{
  "project_id": "uuid",
  "type": "DOCUMENT_GENERATION",
  "revision_id": "uuid",
  "model_version_id": "uuid",
  "parameters": {
    "paper_size": "ARCH_D"
  }
}
```

Response:

```
{
  "data": {
    "job_id": "uuid",
    "status": "QUEUED"
  }
}
```

---

83. JOB STATUS RESPONSE

```
{
  "data": {
    "job_id": "uuid",
    "status": "RUNNING",
    "progress": 47,
    "current_task": "DRAWING_GENERATION",
    "revision_id": "uuid",
    "model_version_id": "uuid"
  }
}
```

---

84. JOB CANCELLATION

`POST /api/v1/jobs/{job_id}/cancel`

The runtime layer verifies that cancellation is permitted.

The response identifies:

```
job_id
status
cancelled_at
```

---

85. JOB EVENT STREAM

Clients may retrieve execution events through:

`GET /api/v1/jobs/{job_id}/events`

Future implementations may provide streaming through:

```
Server-Sent Events
WebSocket
```

without changing the underlying job model.

86. RUNTIME ERROR CONTRACT

Errors use the established OneDraft error structure.

Example:

```
{
  "error": {
    "code": "REVISION_CONFLICT",
    "message": "The requested operation was created against an outdated revision.",
    "resolution": "Refresh the current project revision and submit the operation
again."
  },
  "metadata": {
    "request_id": "uuid",
    "timestamp": "ISO-8601"
  }
}
```

---

87. RUNTIME AUDITABILITY

Every significant runtime action shall be attributable to:

```
actor
service
job
workflow
task
project
revision
```

The resulting execution chain must be reconstructable.

---

88. RUNTIME EVENT IMMUTABILITY

Runtime events are append-only.

The system shall not modify historical execution events to make a workflow appear successful or orderly after the fact.

Corrections are represented through subsequent events.

---

89. DISASTER RECOVERY

The runtime layer shall be recoverable from durable state.

At minimum:

```
jobs
tasks
workflow definitions
workflow executions
events
outbox events
artifact references
```

must survive process-level failure.

---

90. PROCESS RESTART

After a runtime service restart:

```
LOAD QUEUED JOBS
 ↓
LOAD RUNNING JOBS
 ↓
CHECK HEARTBEATS
 ↓
IDENTIFY STALLED TASKS
 ↓
RECOVER RETRYABLE TASKS
 ↓
RESUME WORKFLOWS
```

The objective is continuation rather than silent loss of execution.

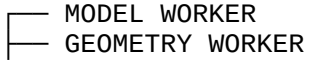
---

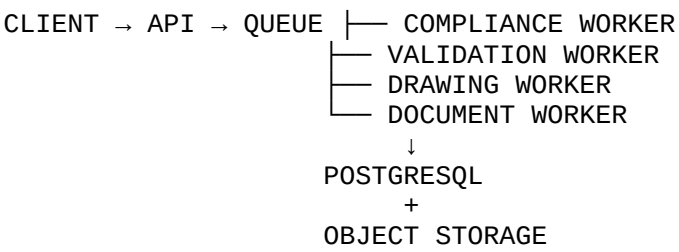
91. DEPLOYMENT MODEL

The initial runtime deployment shall support:

```
API
ORCHESTRATOR
WORKER POOL
POSTGRESQL
QUEUE
OBJECT STORAGE
OBSERVABILITY
```

Conceptually:





---

92. CONTAINERIZATION

Development and deployment should support containerized services.

Initial service boundaries may be represented as:

```
onedraft-api
onedraft-orchestrator
onedraft-worker-model
onedraft-worker-geometry
onedraft-worker-compliance
onedraft-worker-validation
onedraft-worker-document
onedraft-postgres
onedraft-queue
```

The exact deployment topology may evolve without changing the domain contract.

---

93. ENVIRONMENT SEPARATION

Initial environments:

```
DEVELOPMENT
TEST
STAGING
PRODUCTION
```

Runtime jobs shall identify their execution environment.

Production data must not be unintentionally processed by development workers.

---

94. TEST EXECUTION MODE

The orchestrator shall support deterministic test execution.

A test workflow should be able to execute:

```
same project
```

```
same revision
same model version
same workflow version
```

and compare outputs.

---

95. UNIT TESTING

Runtime unit tests shall cover:

```
state transitions
dependency resolution
retry logic
timeout logic
cancellation
version checking
idempotency
progress calculation
authorization
failure classification
```

---

96. INTEGRATION TESTING

Integration tests shall cover:

```
API
 ↓
JOB
 ↓
WORKFLOW
 ↓
WORKER
 ↓
DOMAIN SERVICE
 ↓
DATABASE
```

The test must verify both successful and failed execution paths.

---

97. END-TO-END PROJECT WORKFLOW TEST

The Runtime Orchestrator's primary end-to-end test shall execute:

```
CREATE PROJECT
```

```
 ↓
CREATE REQUIREMENTS
 ↓
CREATE MODEL
 ↓
GENERATE GEOMETRY
 ↓
RUN COMPLIANCE
 ↓
RUN VALIDATION
 ↓
GENERATE DRAWINGS
 ↓
CREATE REDLINE
 ↓
EXECUTE ARIA COMMAND
 ↓
CREATE NEW REVISION
 ↓
REGENERATE AFFECTED DRAWINGS
 ↓
REVALIDATE
 ↓
GENERATE DOCUMENT PACKAGE
 ↓
HASH ARTIFACTS
 ↓
RECORD ACCEPTANCE
 ↓
RECONSTRUCT EXECUTION HISTORY
```

---

98. FAILURE TEST

The system must also intentionally test:

```
worker crash
queue interruption
database interruption
geometry failure
validation failure
drawing failure
document failure
revision conflict
duplicate request
stale job
timeout
cancellation
```

The resulting system must recover without corrupting project state.

---

99. RUNTIME SUCCESS CRITERIA

The Runtime Orchestrator is successful when it can:

receive an authorized operation

create a durable job

construct a workflow

resolve task dependencies

dispatch workers

track execution

enforce project versioning

retry transient failures

reject stale operations

recover stalled tasks

publish execution events

coordinate domain services

produce reproducible artifacts

complete multi-stage workflows

preserve execution history

---

100. PRIMARY RUNTIME WORKFLOW

The canonical execution sequence becomes:

```
REQUEST
  ↓
AUTHENTICATION
  ↓
AUTHORIZATION
  ↓
IDEMPOTENCY
  ↓
JOB
  ↓
WORKFLOW
  ↓
TASK GRAPH
  ↓
WORKERS
  ↓
```

```
DOMAIN SERVICES
 ↓
VALIDATION
 ↓
FINALIZATION
 ↓
ARTIFACTS
 ↓
EVENTS
 ↓
COMPLETION
```

---

101. ONEDRAFT RUNTIME STACK

The OneDraft architecture now operates through the following execution hierarchy:

```
CLIENT
 ↓
API
 ↓
RUNTIME ORCHESTRATOR
 ↓
ARIA / DOMAIN SERVICES
 ↓
PROJECT MODEL
 ↓
GEOMETRY
 ↓
COMPLIANCE
 ↓
VALIDATION
 ↓
REVISION
 ↓
DOCUMENTATION
 ↓
OBJECT STORAGE
```

The Runtime Orchestrator is the connective execution layer between these established systems.

---

102. ARCHITECTURAL BOUNDARY

The critical architectural distinction is:

```
DOMAIN SERVICES
=
WHAT ONE DRAFT KNOWS AND DOES

RUNTIME ORCHESTRATOR
=
WHEN AND HOW ONE DRAFT EXECUTES IT
```

This separation permits individual engines to evolve independently while preserving the overall OneDraft contract.

---

103. ARIA RUNTIME POSITION

Aria operates above the domain services and below the user-facing conversational interface.

The runtime path is:

```
USER
 ↓
ARIA
 ↓
INTENT
 ↓
COMMAND
 ↓
ORCHESTRATOR
 ↓
DOMAIN SERVICES
 ↓
RESULT
 ↓
ARIA
 ↓
USER
```

Aria therefore becomes an intelligent controller of OneDraft capabilities rather than an unrestricted software operator.

---

104. AUTOMATION BOUNDARY

The Runtime Orchestrator enables OneDraft to transition from:

```
REQUEST
→
MANUAL PROCESS
→
RESULT
```

to:

```
REQUEST
→
PLAN
→
EXECUTE
→
VALIDATE
→
```

```
CORRECT
→
REEXECUTE
→
FINALIZE
```

This is the foundation for autonomous workflow execution within controlled application boundaries.

---

105. SELF-HEALING EXECUTION

Runtime recovery may automatically address infrastructure failures.

Examples:

```
worker unavailable
queue retry
temporary storage failure
transient service failure
stale worker
```

However, the orchestrator shall not automatically override:

```
code violations
design constraints
customer decisions
professional review
authorization boundaries
revision conflicts
```

Those require domain resolution or human action.

---

106. AUTONOMOUS DESIGN LOOP

Once the underlying engines are operational, the Runtime Orchestrator permits:

```
DESIGN REQUEST
 ↓
ARIA INTERPRETATION
 ↓
PROPOSAL
 ↓
MODEL MUTATION
 ↓
GEOMETRY
 ↓
COMPLIANCE
 ↓
VALIDATION
 ↓
IF FAILURE
    ↓
```

```
ARIA ANALYSIS
    ↓
CORRECTIVE COMMAND
    ↓
REEXECUTE
 ↓
IF PASS
    ↓
DOCUMENTATION
```

This establishes the execution loop required for the Aria-powered CAD-AI system.

---

107. FINALIZATION GATE

A project workflow may only enter finalization when required gates have passed.

Initial gates:

```
MODEL VALID
GEOMETRY VALID
COMPLIANCE EXECUTED
VALIDATION COMPLETE
DRAWINGS GENERATED
DOCUMENTATION VALID
REQUIRED REVIEW COMPLETE
```

Failure of a mandatory gate prevents finalization.

---

108. ACCEPTANCE GATE

Customer acceptance remains a distinct state.

The runtime workflow becomes:

```
FINAL PACKAGE
 ↓
CUSTOMER REVIEW
 ↓
ACCEPTANCE
```

Acceptance does not retroactively alter the underlying revision.

It records the customer's decision regarding the identified package.

---

109. RECONSTRUCTION REQUIREMENT

Given:

```
project_id
job_id
workflow_execution_id
revision_id
model_version_id
```

the system shall be capable of reconstructing:

```
what was requested
what workflow executed
which tasks executed
which workers processed them
which domain services were called
which version was used
which validation occurred
which artifacts were generated
which failures occurred
how the workflow completed
```

---

110. EXECUTION PROVENANCE

The complete provenance chain becomes:

```
USER REQUEST
 ↓
ARIA INTERPRETATION
 ↓
COMMAND
 ↓
JOB
 ↓
WORKFLOW
 ↓
TASKS
 ↓
MODEL VERSION
 ↓
REVISION
 ↓
CODE MANIFEST
 ↓
VALIDATION
 ↓
DRAWINGS
 ↓
DOCUMENT PACKAGE
 ↓
ACCEPTANCE
```

This chain connects intelligence, execution, design state, regulatory state, documentation and customer outcome.

---

111. FIRST RUNTIME IMPLEMENTATION ARTIFACTS

The next implementation package shall consist of:

```
system/jobs.py
system/workflows.py
system/tasks.py
system/events.py
system/queue.py
system/scheduler.py
system/state_machine.py
system/idempotency.py
system/retry.py
system/worker.py
system/orchestrator.py
```

Database migration artifacts shall include:

```
026_runtime_jobs.sql
027_runtime_tasks.sql
028_workflows.sql
029_runtime_events.sql
030_outbox.sql
031_workers.sql
032_runtime_indexes.sql
```

---

112. FIRST RUNTIME SERVICE

The first executable service shall be:

```
RuntimeOrchestrator
```

Its initial interface shall conceptually provide:

```
create_job()
create_workflow()
dispatch_task()
advance_workflow()
complete_task()
fail_task()
retry_task()
cancel_job()
resume_job()
publish_event()
finalize_job()
```

---

113. FIRST RUNTIME INTEGRATION

The first executable integration shall connect:

```
API
 ↓
RuntimeOrchestrator
 ↓
ProjectModelService
 ↓
PostgreSQL
```

The initial test shall create a project mutation through the API, generate a runtime job, execute the command through the Project Model service, create the resulting revision, and publish the corresponding runtime event.

---

114. SECOND RUNTIME INTEGRATION

The next integration shall connect:

```
RuntimeOrchestrator
 ↓
GeometryService
```

The test shall verify:

```
revision pinning
geometry job creation
worker execution
geometry version verification
result attachment
```

---

115. THIRD RUNTIME INTEGRATION

The next integration shall connect:

```
RuntimeOrchestrator
 ↓
ComplianceService
 ↓
ValidationService
```

The workflow shall verify:

```
code manifest
revision
validation run
validation results
failure gate
```

```
completion gate
```

---

116. FOURTH RUNTIME INTEGRATION

The next integration shall connect:

```
RuntimeOrchestrator
 ↓
DocumentationService
```

The workflow shall execute:

```
MODEL
 ↓
VIEWS
 ↓
DRAWINGS
 ↓
SHEETS
 ↓
DOCUMENT PACKAGE
```

and associate the resulting artifacts with the exact source revision and model version.

---

117. COMPLETE RUNTIME INTEGRATION

The complete OneDraft runtime test becomes:

```
CLIENT
 ↓
API
 ↓
RUNTIME ORCHESTRATOR
 ↓
ARIA
 ↓
PROJECT MODEL
 ↓
GEOMETRY
 ↓
COMPLIANCE
 ↓
VALIDATION
 ↓
REVISION
 ↓
DOCUMENTATION
 ↓
ARTIFACT STORAGE
 ↓
ACCEPTANCE
```

This becomes the primary orchestration regression test.

---

118. VERSION 0.1 ENGINEERING STATE

The OneDraft architecture now contains:

Product Definition

System Architecture

Technology Architecture

Domain Model

Persistence Model

OpenAPI Contract

Project Model

Geometry & Parametric Model Engine

Compliance & Regulatory Engine

Validation Engine

Revision System

Documentation & Drawing Generation Engine

Redline System

Document Provenance

Acceptance Workflow

Runtime Orchestrator

The architecture has therefore progressed from defining the systems to defining how those systems execute together.

---

119. NEXT IMPLEMENTATION ARTIFACTS

The next engineering package shall convert the Runtime Orchestrator specification into executable artifacts:

```
026_runtime_jobs.sql
027_runtime_tasks.sql
028_workflows.sql
029_runtime_events.sql
```

```
030_outbox.sql
031_workers.sql
032_runtime_indexes.sql
```

followed by:

```
runtime_models.py
workflow_engine.py
task_scheduler.py
job_service.py
event_service.py
worker_base.py
runtime_orchestrator.py
```

and corresponding:

```
OpenAPI runtime endpoints
unit tests
integration tests
end-to-end orchestration test
```

---

120. FINAL ARCHITECTURAL DECISION

OneDraft shall not execute as a collection of disconnected services.

It shall operate as a coordinated computational system:

```
PROJECT MODEL
+
ARIA
+
GEOMETRY
+
COMPLIANCE
+
VALIDATION
+
REVISION
+
DOCUMENTATION
+
RUNTIME ORCHESTRATION
```

The Runtime Orchestrator is the execution fabric that turns these individual capabilities into a continuous project lifecycle.

The resulting architecture is:

```
USER
 ↓
ARIA
 ↓
RUNTIME ORCHESTRATOR
 ↓
```

```
┌──────────────────────────┐
│ PROJECT MODEL            │
│ GEOMETRY ENGINE          │
│ COMPLIANCE ENGINE        │
│ VALIDATION ENGINE        │
│ REVISION ENGINE          │
│ DOCUMENTATION ENGINE     │
└──────────────────────────┘
 ↓
VERSIONED PROJECT STATE
 ↓
VALIDATED DOCUMENTATION
 ↓
FINAL DOCUMENT PACKAGE
 ↓
CUSTOMER ACCEPTANCE
```

This establishes the **OneDraft Runtime Execution Layer**.

---

121. SPECIFICATION STATUS

ONEDRAFT RUNTIME ORCHESTRATOR v0.1

STATUS: ESTABLISHED

The domain architecture is now connected to an explicit runtime execution architecture.

The next implementation stage is the conversion of this specification into the executable runtime foundation:

```
026_runtime_jobs.sql
027_runtime_tasks.sql
028_workflows.sql
029_runtime_events.sql
030_outbox.sql
031_workers.sql
032_runtime_indexes.sql
```

followed by the Python orchestration layer and its integration with the existing OneDraft Project Model, Geometry, Compliance, Validation, Revision, and Documentation engines.

The Runtime Orchestrator therefore becomes the execution fabric upon which the remaining OneDraft infrastructure can operate.